

## **Manual De Usuario**

### **Proyecto KQGEO MIP**

**Integrantes:** Jacobo Arroyave Ortiz, Camilo Niño Galvez

**Docente:** Luz Enith Guerrero Mendieta

**Asignatura:** Análisis y Diseño de Algoritmos

**Facultad:** Facultad de Inteligencia Artificial e Ingenierías, Universidad de Caldas

**Período:** 2026-1

**Fecha:** 16 de junio de 2026

**Contenido**

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Manual De Usuario .....                                                                   | 1  |
| Proyecto KQGEO MIP .....                                                                  | 1  |
| 2.1 Introducción y Visión General.....                                                    | 4  |
| ¿Qué hace el software?.....                                                               | 4  |
| ¿Para qué sirve? Aplicaciones prácticas .....                                             | 4  |
| Conceptos clave de forma intuitiva .....                                                  | 4  |
| Estructura general del proyecto.....                                                      | 5  |
| Capacidades y límites de escala del sistema .....                                         | 6  |
| 2.2 Requisitos del Sistema .....                                                          | 7  |
| Sistemas operativos compatibles.....                                                      | 7  |
| Especificaciones de hardware recomendadas .....                                           | 7  |
| Entorno de software requerido .....                                                       | 8  |
| 2.3 Instalación Paso a Paso .....                                                         | 8  |
| Paso 1: Descarga y extracción del proyecto .....                                          | 8  |
| Paso 2: Creación y activación del entorno virtual .....                                   | 8  |
| Paso 3: Instalación de las dependencias requeridas .....                                  | 9  |
| Verificación de la instalación del entorno .....                                          | 10 |
| 2.5 Guía de Uso del Sistema .....                                                         | 11 |
| Perfil de Usuario No Avanzado: Ejecución por CLI e Interfaz Interactiva.....              | 11 |
| <i>Perfil de Usuario Avanzado: Ejecución y Automatización por Lotes desde Excel</i> ..... | 14 |
| 2.6 Opciones y Parámetros Avanzados .....                                                 | 18 |
| Configuración dinámica del pool de candidatos en KQNodes .....                            | 18 |
| Límites del refinamiento local (VND) en KQNodes.....                                      | 19 |
| Configuración de la estrategia Branch and Bound en KGeoMIP .....                          | 19 |
| Modos de operación y límites físicos.....                                                 | 19 |
| Automatización de pruebas masivas en Excel .....                                          | 20 |
| Optimización y consejos de rendimiento.....                                               | 22 |
| 2.7 Solución de Problemas.....                                                            | 22 |
| Errores Comunes en Consola .....                                                          | 22 |
| Problemas de Instalación y Dependencias.....                                              | 23 |
| Problemas de Ejecución y Rendimiento.....                                                 | 24 |
| Diagnóstico de Datos de Entrada Problemáticos.....                                        | 24 |
| 2.9 Referencia Rápida .....                                                               | 24 |
| Comandos de Ejecución Clave.....                                                          | 25 |

|                                              |    |
|----------------------------------------------|----|
| Tabla de Parámetros y Configuraciones.....   | 25 |
| Formato de Archivos de Entrada y Salida..... | 26 |
| Glosario de Términos.....                    | 26 |

## 2.1 Introducción y Visión General

### ¿Qué hace el software?

El software desarrollado en este repositorio es una solución computacional de alto rendimiento en Python diseñada para el análisis estructural, modular y de integración de información en sistemas dinámicos complejos. Su propósito principal es calcular la **Partición de Mínima Información (k-MIP)** de un sistema físico o lógico distribuido en exactamente  $k$  bloques independientes (donde  $2 \leq k \leq 5$ ).

Para resolver este problema, el repositorio integra dos aproximaciones algorítmicas de frontera:

1. **KQNodes**: Extensión del algoritmo Q clásico que minimiza una función submodular de pérdida sobre el espacio de estados mediante fusiones jerárquicas greedy y un refinamiento local de tipo VND (Variable Neighborhood Descent) con swaps. 2. **KGeoMIP**: Implementación geométrica (Método 2) que mapea el espacio de estados en un hipercubo  $n$ -dimensional. Genera una tabla de costos de transición entre estados a través de distancias de Hamming y utiliza un algoritmo Branch and Bound ordenado en cola de prioridad para encontrar la subdivisión de bloques con menor pérdida.

A partir de la Matriz de Probabilidad de Transición (TPM) del sistema, ambos métodos determinan cuál es la partición que minimiza la pérdida de información causal al asumir que los bloques se comportan de forma aislada.

### ¿Para qué sirve? Aplicaciones prácticas

Este software es una herramienta avanzada para investigadores y profesionales en ciencia cognitiva, neurociencia computacional y teoría de redes complejas. Sus aplicaciones clave incluyen:

- **Medición de la Integración de Información (IIT 3.0)**: Permite evaluar el valor de  $\Phi$  (Phi), determinando el nivel de irreducibilidad causal y conciencia causal teórica de circuitos electrónicos y redes biológicas.
- **Identificación de Fronteras Modulares**: Encuentra agrupaciones causales naturales (comunidades) de variables en sistemas dinámicos multivariados.
- **Análisis Causal en Hardware Digital**: Cuantifica el impacto que tienen ciertos componentes o compuertas lógicas sobre el comportamiento de microcontroladores y buses de datos.

### Conceptos clave de forma intuitiva

Para operar la herramienta no se requiere una formación matemática avanzada. Los conceptos esenciales se describen a continuación:

- **Estado del Sistema**: Es la configuración binaria activa (1) o inactiva (0) de todas las variables de la red en un instante. Por ejemplo, en un sistema de 4 variables, el estado "1000" significa que solo la primera variable está encendida.
- **Variables de Presente (Mecanismo) y Futuro (Alcance)**: El análisis causal examina cómo el estado actual de un grupo de variables (mecanismo) determina

probabilísticamente el estado futuro de otro grupo de variables (alcance o purview).

- **$k$ -Partición:** Es una división del sistema de variables en exactamente  $k$  subgrupos disjuntos y no vacíos (bloques). En una tri-partición ( $k = 3$ ) de las variables futuras  $\{A, B, C\}$ , la única división posible es asignar cada variable a su propio bloque:  $\{A\}$ ,  $\{B\}$  y  $\{C\}$ .
- **Pérdida EMD (Earth Mover's Distance):** Es la métrica utilizada para cuantificar la "distancia" o diferencia entre la dinámica del sistema completo y la dinámica simplificada (particionada). Si al desconectar las interacciones entre bloques el sistema se comporta de manera casi idéntica, la pérdida EMD será muy cercana a 0, indicando una frontera natural del sistema.

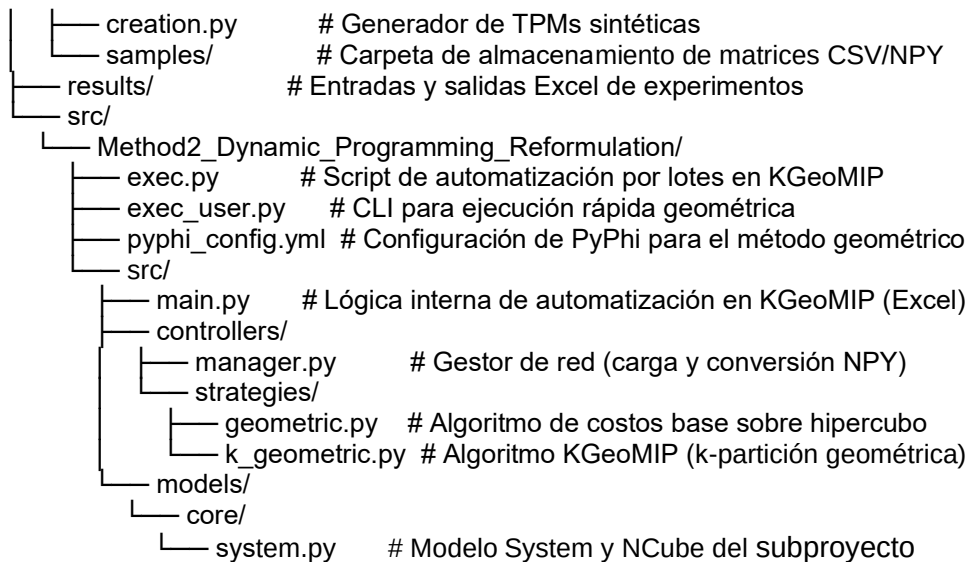
## Estructura general del proyecto

El repositorio completo está estructurado para albergar de forma aislada tanto la base clásica de QNodes como la aproximación geométrica de KGeoMIP, compartiendo la raíz del espacio de trabajo:

```

ADA_2026_1_KQmip/      # Directorio raíz del proyecto
├── main.py             # Interfaz interactiva unificada
├── pyproject.toml      # Configuración de dependencias raíz (uv)
├── uv.lock             # Lockfile de dependencias del entorno uv
├── .doc               # Documentación arquitectónica de bajonivel
├──
├── KQNodes/           # --- Subproyecto Clásico (KQNodes) ---
│   ├── exec.py        # Script de automatización en lote para Excel
│   ├── exec_user.py   # CLI para ejecución rápida de usuario (no avanzado)
│   ├── pyphi_config.yml # Configuración de PyPhi
│   ├── src/
│   │   ├── main.py    # Lógica interna de procesamiento por lotes KQNodes
│   │   ├── controllers/
│   │   │   └── manager.py # Gestor de red (carga de CSV y caché NPY)
│   │   ├── funcs/
│   │   │   ├── force.py # Fuerza bruta y combinatoria
│   │   │   ├── format.py # Visualización y formateo de salidas
│   │   │   └── iit.py    # EMD y utilidades matemáticas IIT
│   │   ├── models/     # Clases de dominio (System, NCube, Solution)
│   │   ├── strategies/
│   │   │   ├── force.py # Búsqueda exhaustiva k-particiones
│   │   │   ├── q_nodes.py # Algoritmo Q clásico (biparticiones)
│   │   │   └── kqnodes/ # Algoritmo KQNodes (k-particiones)
│   │   │       ├── kqnodes.py # Clase orquestadora principal
│   │   │       ├── tree_search.py # Búsqueda de fronteras en árbol de fusiones Q
│   │   │       ├── refinement.py # VND y Swaps de variables
│   │   │       ├── candidates.py # Pools de candidatos heurísticos/estructurales
│   │   │       └── permutation_selector.py # Selector rápido de emparejamientos
│   └── tests/          # Scripts de verificación automatizada
│       ├── test_bruteforce_k3.py # Validación contra fuerza bruta
│       └── test_n25_scalability.py # Test de estrés de velocidad y RAM para N=25
├── KGeoMIP/           # --- Subproyecto Geométrico (GeoMIP) ---
├── Dataset_Description.md # Descripción técnica de los datasets TPM
└── data/

```



## Capacidades y límites de escala del sistema

Ambos subproyectos adaptan dinámicamente sus estrategias según el tamaño del sistema para garantizar el mejor compromiso entre exactitud matemática y tiempo de procesamiento:

### A. Estrategia KQNodes

1. **Sistemas Pequeños ( $N \leq 2$  variables de futuro):** Ejecuta una **Búsqueda Exhaustiva Exacta (Fuerza Bruta)**. Evalúa todas las posibles  $k$ -particiones posibles (números de Stirling del segundo tipo) en milisegundos y garantiza encontrar el mínimo global absoluto.
2. **Sistemas Medianos a Grandes ( $7 \leq N \leq 22$  variables):** Emplea el algoritmo **KQNodes**. Construye un árbol de fusiones jerárquicas submodulares (árbol Q) para enfocar la búsqueda en las fronteras de menor pérdida, y luego aplica una **Búsqueda Local de Vecindario Variable (VND)** para refinar la solución. Encuentra la solución en menos de un segundo, reduciendo exponencialmente el tiempo frente a la Fuerza Bruta.
3. **Sistemas Masivos ( $N > 22$  variables):** El programa activa un **Bypass estructural automático**. Salta la construcción costosa del árbol Q en memoria y genera una partición inicial instantánea basada en el acoplamiento directo de variables, refinándola localmente. Esto permite calcular tri-particiones de sistemas masivos de 25 variables ( $N = 25$ , que representan 33,554,432 estados de transición) en tan solo **0.71 segundos** sin agotar la memoria de la máquina (evitando errores OOM).

### B. Estrategia KGeoMIP

1. **Sistemas Pequeños y Medianos ( $N < 20$  variables):** Calcula de forma exacta el recorrido bottom-up de transiciones en todos los niveles Hamming del hipercubo. Proporciona una tabla de costos muy detallada que permite evaluar la separación de bloques a través de la frontera ordenada de ramificación y acotación (Branch and Bound).
2. **Sistemas Grandes ( $N \geq 20$  variables):** Activa de forma automática el **Modo Grande**. Restringe el cálculo recursivo de la tabla de costos únicamente al primer nivel Hamming

(distancia 1). Esto reduce drásticamente el uso de memoria RAM y CPU al evitar la explosión combinatoria de estados intermedios en el hipercubo, manteniendo una precisión óptima en la estimación de la pérdida.

## 2.2 Requisitos del Sistema

Enlace a video tutorial: <https://youtu.be/zhhhovgx7A>

Enlace Repositorio Github: [https://github.com/CamiloNinoG/ADA\\_2026\\_1\\_KQmip](https://github.com/CamiloNinoG/ADA_2026_1_KQmip)

### Sistemas operativos compatibles

El software está escrito en código de Python estándar y es compatible con los siguientes entornos:

- **Windows (Entorno recomendado):** Windows 10 y Windows 11 de 64 bits.
- **Linux:** Ubuntu 20.04 LTS o superior, Debian, CentOS o RedHat.
- **macOS:** macOS 12 (Monterey) o superior (compatible con arquitecturas Intel y Apple Silicon).

### Especificaciones de hardware recomendadas

| Recurso                 | Requisitos Mínimos<br>(Sistemas $N \leq 15$ )    | Requisitos Recomendados<br>(Sistemas $15 < N \leq 25$ )                                 |
|-------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Procesador (CPU)</b> | Intel Core i3 o AMD Ryzen 3 (2 núcleos, 2.0 GHz) | Intel Core i5/i7 o AMD Ryzen 5/7 (4+ núcleos, 3.2 GHz o más)                            |
| <b>Memoria RAM</b>      | 4 Gigabytes (GB)                                 | 8 Gigabytes (GB) a 16 Gigabytes (GB) para evitar swapping con redes de $N = 25$         |
| <b>Almacenamiento</b>   | 100 Megabytes (MB) de espacio disponible         | 2 Gigabytes (GB) de espacio disponible (necesario para albergar matrices .npy gigantes) |

[!WARNING] **Sobre la carga de matrices masivas:** La carga en memoria de matrices de probabilidad de transición (TPM) para redes de 25 variables (como N25A.npy) requiere aproximadamente 838 MB de almacenamiento en disco en formato binario. Al cargarse en NumPy, el sistema operativo requiere memoria física libre suficiente para evitar que la máquina empiece a escribir en el disco duro (swapping), lo cual degradaría drásticamente los tiempos de procesamiento.

### Entorno de software requerido

- **Python:** Versión **3.11** o **3.12** de 64 bits instalada en el sistema.
  - **Administrador de Paquetes:** pip (incluido por defecto en Python) o la herramienta moderna de alto rendimiento **uv**.
- 

## 2.3 Instalación Paso a Paso

### Paso 1: Descarga y extracción del proyecto

1. Obtenga el repositorio del proyecto en su máquina local. Si cuenta con el cliente de Git configurado en su consola, ejecute:

```
▲ jacobito ~/descargas >> git clone https://github.com/CamiloNinoG/ADA_2026_1_KQ_NODES
cd ADA_2026_1_KQ_NODES

Clonando en 'ADA_2026_1_KQ_NODES'...
remote: Enumerating objects: 503, done.
remote: Counting objects: 100% (222/222), done.
remote: Compressing objects: 100% (131/131), done.
remote: Total 503 (delta 92), reused 202 (delta 88), pack-reused 281 (from 2)
Recibiendo objetos: 100% (503/503), 65.92 MiB | 1013.00 KiB/s, listo.
Resolviendo deltas: 100% (156/156), listo.
▲ jacobito ~/descargas/ADA_2026_1_KQ_NODES git-[v main]- >>
```

2. Si cuenta con un archivo comprimido (.zip o .tar.gz), descomprímalo en un directorio local de su preferencia, por ejemplo:

C:\Users\SuUsuario\Proyectos\ADA\_2026\_1\_KQ\_NODES

### Paso 2: Creación y activación del entorno virtual

El uso de un entorno virtual garantiza que las librerías del proyecto no interfieran con otras instalaciones de Python en su computadora. Abra una terminal de comandos (PowerShell en Windows, o Terminal en Linux/macOS) en el directorio raíz del proyecto y elija uno de los siguientes métodos:

#### **Opción A: Usando Python estándar (Flujo clásico)**

En Windows (PowerShell):

```
# 1. Crear el entorno virtual en la carpeta .venv
python -m venv .venv
# 2. Activar el entorno virtual en su sesión de PowerShell
.venv\Scripts\Activate.ps1
```

En Linux o macOS:

```
# 1. Crear el entorno virtual
python3 -m venv .venv
```



# 2. Activar el entorno virtual

`source .venv/bin/activate`

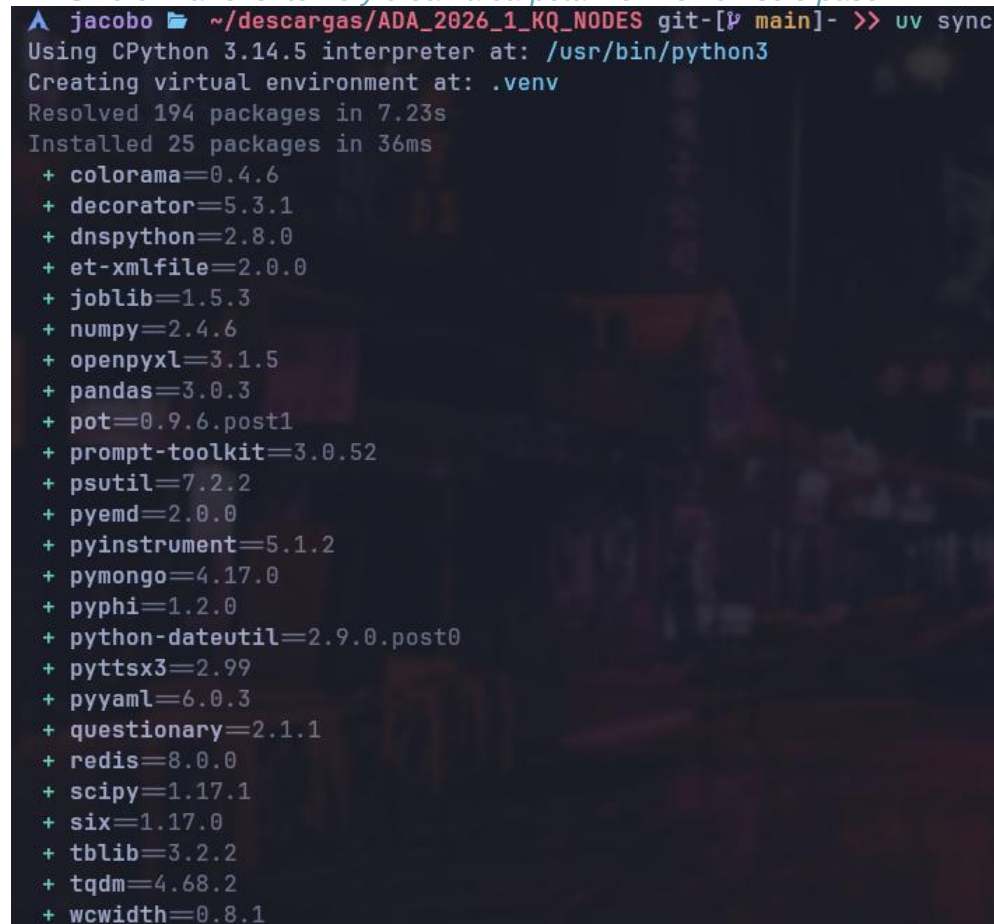
### **Opción B: Usando la herramienta ultra rápida uv (Recomendado)**

uv es un administrador de entornos y paquetes desarrollado en Rust extremadamente rápido. Si decide usarlo:

# 1. Instalar uv a nivel global o de usuario (Si no tiene instalado uv )

`pip install uv`

# 2. Sincronizar el entorno y crear la carpeta .venv en un solo paso



```
└─ jacobol ~/descargas/ADA_2026_1_KQ_NODES git-[main]- >> uv sync
Using CPython 3.14.5 interpreter at: /usr/bin/python3
Creating virtual environment at: .venv
Resolved 194 packages in 7.23s
Installed 25 packages in 36ms
+ colorama=0.4.6
+ decorator=5.3.1
+ dnspython=2.8.0
+ et-xmlfile=2.0.0
+ joblib=1.5.3
+ numpy=2.4.6
+ openpyxl=3.1.5
+ pandas=3.0.3
+ pot=0.9.6.post1
+ prompt-toolkit=3.0.52
+ psutil=7.2.2
+ pyemd=2.0.0
+ pyinstrument=5.1.2
+ pymongo=4.17.0
+ pyphi=1.2.0
+ python-dateutil=2.9.0.post0
+ pyttsx3=2.99
+ pyyaml=6.0.3
+ questionnaire=2.1.1
+ redis=8.0.0
+ scipy=1.17.1
+ six=1.17.0
+ tblib=3.2.2
+ tqdm=4.68.2
+ wcwidth=0.8.1
```

### Paso 3: Instalación de las dependencias requeridas

Si optó por la **Opción A (Python estándar)**, con el entorno virtual activado, ejecute el siguiente comando en la consola para instalar las librerías necesarias:

# Asegurar que pip esté actualizado

`python -m pip install --upgrade pip`

# Instalar los paquetes clave de análisis numérico e interfaz

`pip install numpy pandas openpyxl colorama pyinstrument questionnaire`

**Detalle de las dependencias instaladas:**

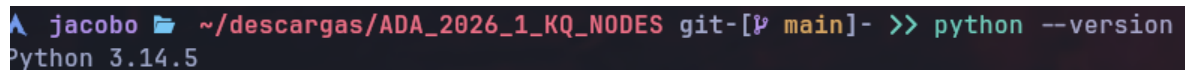
- **Numpy** (v1.24+): Manejo algebraico de matrices de probabilidad e implementación de la distancia EMD.
- **Pandas** (v2.0+): Lectura estructurada de filas y celdas del archivo de origen de Excel.
- **Openpyxl** (v3.1+): Escritura directa sobre plantillas XLSX manteniendo formatos sin dañar el archivo original.  
colorama: Habilita la salida con colores ANSI en terminales de comandos de Windows.
- **pyinstrument**: Profiler utilizado para registrar cuellos de botella y perfilar los tiempos exactos de cómputo. \* **questionary**: Generación de menús interactivos interactivos por consola.

**Verificación de la instalación del entorno**

Antes de ejecutar el solver, verifique que el entorno virtual esté activo y las librerías se hayan enlazado correctamente con sus versiones recomendadas. Ejecute los siguientes comandos de diagnóstico en su terminal:

**1. Verificar la versión de Python activa:**

```
python --version
```



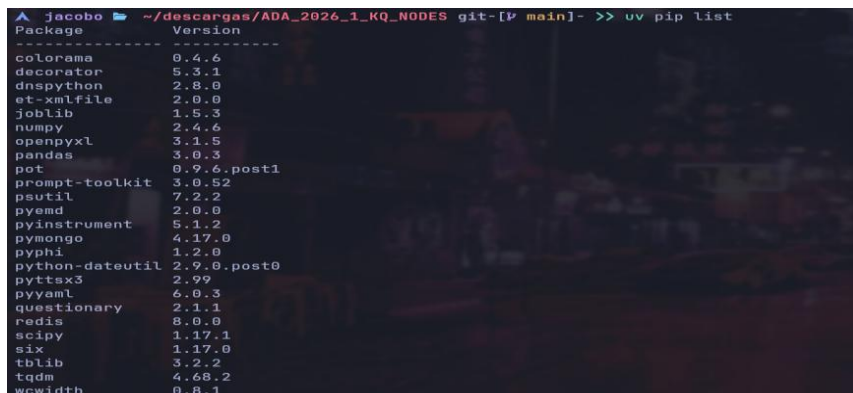
```
^ jacoboj ~/descargas/ADA_2026_1_KQ_NODES git-[P main]- >> python --version
Python 3.14.5
```

**2. Listar paquetes instalados:** Si usó Python estándar:

```
pip list
```

Si usó la herramienta uv:

```
uv pip list
```



```

jacob@ ~/descargas/ADA_2026_1_KQ_NODES git-[P main]- >> uv pip list
Package Version
-----
colorama 0.4.6
decorator 5.3.1
dnspython 2.8.0
et-xmlfile 2.0.0
joblib 1.5.3
numpy 2.4.6
openpyxl 3.1.5
pandas 3.0.3
pot 0.9.6.post1
prompt-toolkit 3.0.52
psutil 7.2.2
pyemd 2.0.0
pyinstrument 5.1.2
pymongo 4.17.0
pyphi 1.2.0
python-dateutil 2.9.0.post0
pytsx3 2.99
pyyaml 6.0.3
questionnaire 2.1.1
redis 8.0.0
scipy 1.17.1
six 1.17.0
tblib 3.2.2
tqdm 4.68.2
wcwidth 0.8.1

```

## 2.5 Guía de Uso del Sistema

El framework **K-MIP/IIT** ofrece dos perfiles de uso claramente diferenciados según las necesidades técnicas y el nivel de familiaridad del usuario con la codificación en Python:

### Perfil de Usuario No Avanzado: Ejecución por CLI e Interfaz Interactiva

Este perfil está diseñado para usuarios que desean evaluar subsistemas de forma directa sin necesidad de editar código fuente en Python o configurar archivos de automatización estructurados.

#### A. Interfaz Consola Interactiva (Recomendada)

El archivo unificado `main.py` ubicado en la raíz del repositorio actúa como un asistente guiado interactivo.

1. Abra una terminal de comandos en la carpeta raíz del repositorio.
2. Inicie el asistente interactivo:

```
uv run main.py
```

(O `.venv/bin/python main.py` si utiliza el flujo clásico).

3. El asistente le pedirá interactuar con una serie de preguntas utilizando las flechas del teclado y la tecla Enter:
  - **Algoritmo:** Seleccione QNodes (Algoritmo Q) o KGeoMIP (Método Geométrico).
  - **Tamaño de la TPM:** Seleccione la dimensión del sistema (10, 15, 20, 22, 25).
  - **Valor de k:** Defina en cuántos bloques subdividir el subsistema (2, 3, 4, 5).

- **Estado Inicial:** Seleccione entre las opciones automáticas (Todo en 0, 1 seguido de ceros) o digite manualmente el vector de bits.
  - **Condiciones / Alcance / Mecanismo:** Defina las máscaras seleccionando Todo en 1 o ingrese el vector binario manualmente.
4. El asistente llamará automáticamente al algoritmo seleccionado y mostrará el objeto solución final con la partición formateada en la terminal.

```
? Seleccione el algoritmo: (Use arrow keys)
» KGeoMIP
  QNodes
? Seleccione el algoritmo: (Use arrow keys)
» KGeoMIP
  QNodes
? Seleccione el algoritmo: KGeoMIP
? Seleccione el tamaño de la TPM: 10
? Seleccione k: 2
? Seleccione el estado inicial: 1 seguido de ceros
? Seleccione las condiciones: Todo en 1
? Seleccione el alcance: Todo en 1
? Seleccione el mecanismo: Todo en 1
1000000000
1111111111
1111111111
1111111111
2
10
Tamaño estimado: 0.000010 GB
Tiempo estimado: 0.0 segundos
Generando estados...
Generación completada en 0.00 segundos
Guardando en src/.samples/N10A.csv...
Versión binaria optimizada guardada en N10A.npy
Archivo guardado: 0.000019 GB
Tiempo de guardado: 0.00 segundos
[20:48:30] [ ] Cargando caché binario ultra rápido: N10A.npy
```

### **B. Ejecución CLI Directa (exec\_user.py)**

Tanto el subproyecto QNodes como GeoMIP/src/Method2\_Dynamic\_Programming\_Reformulation contienen un script llamado exec\_user.py. Estos scripts permiten invocar los algoritmos directamente mediante parámetros de línea de comandos (flags), ideal para automatizaciones externas breves o integración rápida en bash.

Las banderas disponibles para ambos scripts son: \* --estado: Vector binario del estado inicial (ej. 1000000000). \* --condiciones: Vector binario de condiciones de fondo (ej. 1111111111). \* --alcance: Máscara binaria del alcance en el futuro (ej. 1101101101). \* --mecanismo: Máscara binaria del mecanismo en el presente (ej. 1101101101). \* --k: Entero con la cantidad de particiones a evaluar (2,3,4,5). \* --tamano: Dimensión de la TPM a cargar.

### Ejemplo de ejecución para QNodes:

```
cd KQNodes
```

```
uv run exec_user.py --estado 1000000000 --condiciones 1111111111 --alcance 1101101101 --mecanismo 1101101101 --k 3 --tamano 10
```

```

-----
A jacobso ~/descargas/ADA_2026_1_KQ_NODES git-[V main]- >> cd QNodes
A jacobso ~/descargas/ADA_2026_1_KQ_NODES/QNodes git-[V main]- >> uv run exec_user.py --estado 1000000000 --condiciones 1111111111 --alcance 1101101101 --mecanismo 1101101101 --k 3 --tamano 10
1000000000
1111111111
1101101101
1101101101
3
10
Tamaño estimado: 0.000010 GB
Tiempo estimado: 0.0 segundos
Generando estados...
Generación completada en 0.00 segundos
Guardando en src/.samples/N10A.csv...
Versión binaria optimizada guardada en N10A.npy
Archivo guardado: 0.000019 GB
Tiempo de guardado: 0.00 segundos
[21:01:30] [] Cargando caché binario ultra rápido: N10A.npy
CRITICAL (21:01:30): Sistema Candidato creado.
CRITICAL (21:01:30): Subistema creado.
→ Candidatos totales en pool pre-ordenados: 30
[Refinamiento] Pérdida antes: 0.328125
PASO REFINEMENT: 0
PASO REFINEMENT: 1
PASO REFINEMENT: 2
PASO REFINEMENT: 3
[Refinamiento] Pérdida después de Búsqueda Local: 0.261719

KQNodes(k=3) fue la estrategia de solución.

Distancia métrica utilizada:
distancia-hamming
Notación utilizada en indexación:
little-endian

Distribucion marginal del Subistema:
[ 0.2500 0.6250 0.5000 0.3750 0.5000 0.5000 0.6250 ]
Distribucion marginal de la Partición:
[ 0.4375 0.5625 0.5000 0.3750 0.4941 0.5059 0.6250 ]

Mejor Bi-Partición:
( G ) ( H ) ( A R R F )

```

### Ejemplo de ejecución para KGeoMIP:

```
cd KGeoMIP/src/Method2_Dynamic_Programming_Reformulation
```

```
uv run exec_user.py --estado 1000000000 --condiciones 1111111111 --alcance 1101101101 --mecanismo 1101101101 --k 3 --tamano 10
```

```

jacob@ ~/descargas/ADA_2026_1_KQ_NODES/GeoMIP/src/Method2_Dynamic_Programming_Reformulation git-[v main]- >> uv run exec_user.py --esta
ance 1101101101 --mecanismo 1101101101 --k 3 --tamano 10
1000000000
1111111111
1101101101
1101101101
3
10
Tamaño estimado: 0.000010 00
Tiempo estimado: 0.0 segundos
Ya existe N10A.csv. ¿Generar nueva red? (s/n): n
[21:02:04] [] Cargando caché binario ultra rápido: N10A.npy
CRITICAL (21:02:06): Original creado.
CRITICAL (21:02:06): Original:
CRITICAL (21:02:06): Candidato creado.
CRITICAL (21:02:06): Subsistema creado.
CRITICAL (21:02:06): empieza.
Modo pequeño activado: calculando costos para todos los niveles
modo pequeño con 7 tamaño
modo pequeño con 7 tamaño
modo pequeño con 7 tamaño

K-GeoMIP fue la estrategia de solución.

Distancia métrica utilizada:
end-effect
Notación utilizada en indexación:
little-endian

Distribucion marginal del Subsistema:
[ 0.7500 0.3750 0.5000 0.6250 0.5000 0.5000 0.3750 ]
Distribucion marginal de la Partición:
[ 0.7500 0.3750 0.5000 0.6250 0.4912 0.4834 0.3750 ]

Mejor Bi-Partición:
[ [ 0 ] ]
[ [ 0 ] ] = [ [ H ] ]
[ [ 0 ] ] = [ [ A,B,D,E,J ] ]
[ [ 0,b,d,e,g,h,j ] ]
Pérdida mínima (φ) = 0.0254

Tiempos de ejecución:

```

## Perfil de Usuario Avanzado: Ejecución y Automatización por Lotes desde Excel

Este perfil está destinado a investigadores que necesitan ejecutar análisis masivos sobre plantillas estructuradas de Excel, requiriendo modificar directamente scripts internos para configurar comportamientos lógicos personalizados.

### A. Automatización de pruebas por lotes en QNodes

La ejecución por lotes de QNodes lee la configuración de subsistemas mapeados por letras (ej: "A,B,C") desde un Excel de entrada e inyecta la partición mínima, pérdida y tiempo por cada valor de  $k$  ( $2 \leq k \leq 5$ ).

- **Archivo Excel de entrada:** KQNodes/src/results/pruebas.xlsx
- **Archivo Excel de salida:** KQNodes/src/results/pruebas\_con\_resultados.xlsx
- **Procedimiento:**
  1. Abra el editor y navegue a KQNodes/src/main.py.
  2. Modifique la constante LONGITUD\_ELEMENTOS en la línea 15 para definir el tamaño del sistema a evaluar (ej. LONGITUD\_ELEMENTOS = 10 cargará la hoja 10A-Elementos).



```

1  import os
2  import time
3  import pandas as pd
4  from datetime import datetime
5  import openpyxl
6  from src.controllers.manager import Manager
7  from src.strategies.qnodes.qnodes import QNodes
8  from src.strategies.q_nodes import QNodes
9  from src.strategies.force import BruteForce
10 from src.strategies.qnodes.profiler import QNodesProfiler
11
12 #
13 # CONFIGURACIÓN DINÁMICA
14 #
15 LONGITUD_ELEMENTOS = 10  Camilo Nifo, 9 hours ago via PR #1 · implementación final con test
16 nombre_hoja = f"(LONGITUD_ELEMENTOS)A-Elementos"
17 # Método a usar
18 estado_inicial = "1" + "0" * (LONGITUD_ELEMENTOS - 1) # Ej: "1000000000" para 10b
19 condiciones = "1" * LONGITUD_ELEMENTOS # Ej: "1111111111" para 10b
20 gestor_redes = Manager(estado_inicial)
21 mpt = gestor_redes.cargar_red()
22 analizador_bf = QNodes(mpt) # "QNodes", "QNodes", "BruteForce"
23
24
25 #
26 def obtener_timestamp():
27     """Devuelve la hora actual formateada para los prints."""
28     return datetime.now().strftime("%H:%M:%S.%f")[:-3]
29
30

```

3. Ajuste el rango de particiones si desea limitar el procesamiento (línea 97: for k in range(2, 6)).

```

41 def iniciar():
42     pruebas_procesadas += 1
43     num_prueba = int(id_prueba) if not pd.isna(id_prueba) else (index + 1)
44     fila_excel = index + 6
45     alcance_bin = texto_a_binario(alcance_txt, universo_letras)
46     mecanismo_bin = texto_a_binario(mecanismo_txt, universo_letras)
47     print(
48         f"[(obtener_timestamp())] @ [Prueba #{num_prueba}] (Fila Excel: {fila_excel})"
49     )
50     print(
51         f"Datos -> Alcance: {alcance_txt} ({alcance_bin}) | Mecanismo: {mecanismo_txt} ({mecanismo_bin})"
52     )
53     # LOOP SOLICITADO: De K = 3 hasta 4 (K=3) para la misma fila
54     for k in range(2, 6):  Camilo Nifo, 9 hours ago via PR #1 · implementación final con test
55         print(f"    Ejecutando 'aplicar_estrategia_k' con K = {k}...")
56         tiempo_inicio = time.time()
57         try:
58             analizador_bf.reset_estado()
59             sia_cero = analizador_bf.aplicar_estrategia_k(
60                 estado_inicial,
61                 condiciones,
62                 alcance_bin,
63                 mecanismo_bin,
64                 k, # Inyección dinámica de K
65             )
66             tiempo_fin = time.time() - tiempo_inicio
67             particion_val = getattr(sia_cero, "particion", "N/A")
68             perdida_val = getattr(
69                 sia_cero, "phi", getattr(sia_cero, "perdida", sia_cero)
70             )
71             tiempo_str = f"(tiempo fin: {tiempo_fin})"

```

4. Ubicado en la carpeta QNodes, ejecute el script:

uv run [exec.py](#)

```

A jacob@ ~ - ssh - 7.35 GiB 78.10 GB 192.168.0.10
~/descargas/ADA_2026_1_KQ_NODES/QNodes git-[V main]- >> uv run exec.py
[21:09:18] [ ] Cargando caché binario ultra rápido: NIBA.npy
[21:09:18.639] [ ] Iniciando proceso de automatización estructurado...
[21:09:18.639] [ ] Cargando datos con Pandas para el procesamiento lógico...
[21:09:18.722] [ ] Total de filas detectadas por Pandas: 14
[21:09:18.722] [ ] Abriendo plantilla con openpyxl...
[21:09:19.515] [ ] Inicializando Gestor de Redes (Manager) a 10 bits...
[21:09:19.515] [ ] Inicializando Analizador de (KQNodes)...

[21:09:19.516] [ ] Procesando e inyectando resultados...
=====
[21:09:19.519] [ ] [Prueba #33] (Fila Excel: 6)
[ ] Datos -> Alcance: ABDEGHJ (1101101101) | Mecanismo: ABDEGHJ (1101101101)
+ Ejecutando 'aplicar_estrategia_k' con K = 2...
CRITICAL (21:09:19): Sistema Candidato creado.
CRITICAL (21:09:19): Subistema creado.
[Exhaustivo] Ejecutando búsqueda exhaustiva para N=7 (futuro) y N_pres=7 (presente) | Combinaciones: 8864 | k=2...
+ [OK K=2] 1.4036s | Partición: { G } { A,B,D,E,H,J }
{ d } { a,b,d,e,g,h,j }
| Pérdida: 0.0087890625
+ Ejecutando 'aplicar_estrategia_k' con K = 3...
CRITICAL (21:09:20): Sistema Candidato creado.
CRITICAL (21:09:20): Subistema creado.
+ Candidatos totales en pool pre-ordenados: 30
[Refinamiento] Pérdida antes: 0.328125
PASO REFINEMENT: 0
PASO REFINEMENT: 1
PASO REFINEMENT: 2
PASO REFINEMENT: 3
[Refinamiento] Pérdida después de Búsqueda Local: 0.261719
+ [OK K=3] 0.0491s | Partición: { G } { H } { A,B,D,E,H,J }
{ d } { a } { b,e,g,h,j }
| Pérdida: 0.26171875
+ Ejecutando 'aplicar_estrategia_k' con K = 4...
CRITICAL (21:09:20): Sistema Candidato creado.
CRITICAL (21:09:20): Subistema creado.
+ Candidatos totales en pool pre-ordenados: 25
[Refinamiento] Pérdida antes: 0.566406
PASO REFINEMENT: 0
PASO REFINEMENT: 1
PASO REFINEMENT: 2
PASO REFINEMENT: 3
[Refinamiento] Pérdida después de Búsqueda Local: 0.541016

```

- Al finalizar, consulte el archivo de salida generado, que contendrá todas las métricas inyectadas de forma nativa.

| pruebas_con_resultados.xlsx                                     |               |                         |                       |                                              |              |         |                                     |
|-----------------------------------------------------------------|---------------|-------------------------|-----------------------|----------------------------------------------|--------------|---------|-------------------------------------|
| QNodes > src > results > pruebas_con_resultados.xlsx            |               |                         |                       |                                              |              |         |                                     |
| 10A-Elementos                                                   |               |                         |                       |                                              |              |         |                                     |
| Edit Table Expand Plain Settings Refresh Undo Redo Convert Link |               |                         |                       |                                              |              |         |                                     |
|                                                                 | B             | C                       | D                     | E                                            | F            | G       | H                                   |
| 2                                                               | ia:           | ABCDEFGHJI              |                       |                                              |              |         |                                     |
| 3                                                               | ia Candidato: | ABCDEFGHJI              |                       |                                              |              |         |                                     |
| 4                                                               |               | Subsistema              | PRUEBAS BIPARTICIONES |                                              |              |         | Geometric                           |
| 5                                                               | a             | Alcance o Purview (t+1) | Mecanismo(t)          | Partición                                    | Pérdida      | Tiempo  | Partición                           |
| 6                                                               |               | ABDEGHJ                 | ABDEGHJ               | ( G ∨ A,B,D,E,H,J )<br>( ∅ ∧ a,b,d,e,g,h,j ) | 0.0087890625 | 1.4036s | ( G ∨ H ∨ A,B,D,E,J ) \ d ∧ ...     |
| 7                                                               |               | ACEGI                   | ACEGI                 | ( A ∨ C,E,G,I )<br>( ∅ ∧ a,c,e,g,i )         | 0.0146484375 | 0.0584s | ( A ∨ C ∨ E,G,I ) \ ( ∅ ∧ ∅ ) ...   |
| 8                                                               |               | BDFHJ                   | BDFHJ                 | ( F ∨ B,D,H,J )<br>( ∅ ∧ b,d,f,h,j )         | 0.0009765625 | 0.0581s | ( F ∨ H ∨ B,D,J ) \ ( ∅ ∧ ∅ ) ...   |
| 9                                                               |               | ABDEGHJ                 | ACEGI                 | ( A ∨ B,D,E,G,H,J )<br>( ∅ ∧ a,c,e,g,i )     | 0.0146484375 | 0.3128s | ( A,D,E ∨ B,G,H ∨ J ) \ a,e,...     |
| 10                                                              |               | ABDEGHJ                 | BDFHJ                 | ( G ∨ A,B,D,E,H,J )<br>( ∅ ∧ b,d,f,h,j )     | 0.0087890625 | 0.3137s | ( A,E,J ∨ B,G,H ∨ D ) \ d,h ...     |
| 11                                                              |               | ACEGI                   | ABDEGHJ               | ( G ∨ A,C,E,I )<br>( ∅ ∧ a,b,d,e,g,h,j )     | 0.0087890625 | 0.2608s | ( G ∨ A,C,E ∨ I ) \ ( ∅ ∧ b,e,...   |
| 12                                                              |               | ACEGI                   | BDFHJ                 | ( G ∨ A,C,E,I )<br>( ∅ ∧ b,d,f,h,j )         | 0.0087890625 | 0.0583s | ( C ∨ G ∨ A,E,I ) \ ( ∅ ∧ ∅ ) ...   |
| 13                                                              |               | BDFHJ                   | ABDEGHJ               | ( F ∨ B,D,H,J )<br>( ∅ ∧ a,b,d,e,g,h,j )     | 0.0009765625 | 0.2590s | ( B,D,J ∨ F ∨ H ) \ ( b,e,g,h,j ... |
| 14                                                              |               | BDFHJ                   | ACEGI                 | ( B ∨ D,F,H,J )<br>( ∅ ∧ a,c,e,g,i )         | 0.021484375  | 0.0583s | ( B ∨ F ∨ D,H,J ) \ ( ∅ ∧ ∅ ) ...   |



## ***B. Automatización de pruebas por lotes en KGeoMIP***

El script de KGeoMIP procesa múltiples filas de un archivo Excel de forma concurrente mediante multiprocessing en Python (aislando la ejecución de cada subsistema en un proceso con timeout para evitar cuellos de botella en redes muy grandes).

- **Archivo Excel de entrada/salida:** GeoMIP/results/DatosPruebas2026\_1.xlsx
- **Procedimiento:**
  1. Abra el editor y navegue a  
GeoMIP/src/Method2\_Dynamic\_Programming\_Reformulation/src/main.py.
  2. Modifique la función iniciar() al final del archivo con las siguientes consideraciones:
    - range (2,6) es el rango de k para las pruebas en este caso las pruebas se harán desde 2 particiones hasta 5
    - ruta\_excel es la ruta de excel que servirá como entrada de datos y salida de datos
    - Hoja define el número de hoja donde se tiene la información de entrada y donde se escribirán los resultados del sistema de tamaño n
    - Cantidad representa el número de filas con que se probará el sistema donde cada fila es una entrada de datos diferentes ya que representa un mecanismo y un alcance
    - Tamaño\_estado es el número de variables que tendrá el sistema en el futuro

```
def iniciar():  
    ruta_excel = Path(  
        os.getenv(  
            "GEOMIP_INPUT_XLSX",  
            str(GEOMIP_ROOT / "results" / "DatosPruebas2026_1.xlsx"),  
        )  
    )  
    for k in range(2, 6):  
        ejecutar_desde_excel(  
            ruta_excel=ruta_excel,  
            hoja=5,  
            k=k,  
            tamaño_estado=25,  
            cantidad=50  
        )
```

3. Modifique columnas teniendo en cuenta que en cada llave representa donde se escribirá los resultados en el excel excel partición perdida y tiempo para cada subsistema con partición k

```
COLUMNAS = {  
    2: {"particion": "G", "perdida": "H", "tiempo": "I"},  
    3: {"particion": "M", "perdida": "N", "tiempo": "O"},  
    4: {"particion": "S", "perdida": "T", "tiempo": "U"},  
    5: {"particion": "Y", "perdida": "Z", "tiempo": "AA"},  
}
```

4. El script actualizará de forma directa las celdas del Excel de origen y mostrará los resultados intermedios por terminal ubicado en el subdirectorio GeoMIP/src/Method2\_Dynamic\_Programming\_Reformulation, ejecute el script:

uv run [exec.py](#)

---

## 2.6 Opciones y Parámetros Avanzados

Esta sección está destinada a usuarios e investigadores que requieren ajustar el comportamiento interno del algoritmo, ejecutar pruebas masivas mediante plantillas de Excel, o realizar optimizaciones para sistemas de dimensiones específicas.

### Configuración dinámica del pool de candidatos en KQNodes

En el archivo [candidates.py](#), el software adapta dinámicamente el tamaño de la búsqueda heurística según el número de variables futuras  $N$  para mantener una alta velocidad sin perder precisión causal. Estos parámetros controlan la generación de candidatos:

- **top\_q**: Número máximo de candidatos de corte extraídos del historial de fusiones del Algoritmo Q (árbol de fusiones).
- **top\_struct**: Número de configuraciones estructurales heurísticas generadas por balanceo cíclico y cercanía.
- **max\_pairs**: Número máximo de combinaciones cruzadas permitidas en el pool.
- **tope**: Límite superior de candidatos de emparejamiento evaluados en la búsqueda de local.

### Valores dinámicos por defecto KQNodes :

- **Sistemas Grandes ( $N \geq 20$ )**: top\_q = 2, top\_struct = 2, max\_pairs = 4, tope = 3.
  - *Razón*: Minimiza la sobrecarga computacional en redes con millones de estados.

- **Sistemas Medianos** ( $15 \leq N < 20$ ):  $\text{top\_q} = 4$ ,  $\text{top\_struct} = 3$ ,  $\text{max\_pairs} = 12$ ,  $\text{tope} = 30$ .
- **Sistemas Pequeños** ( $N < 15$ ):  $\text{top\_q} = 8$ ,  $\text{top\_struct} = 5$ ,  $\text{max\_pairs} = 30$ ,  $\text{tope} = 50$ .

### Límites del refinamiento local (VND) en KQNodes

La Búsqueda Local de Vecindario Variable (VND) en [refinement.py](#) optimiza la partición seleccionada aplicando dos vecindarios de movimientos (Shift e Intercambio/Swap). Su presupuesto se escala automáticamente en función de la cantidad de variables futuras  $N$ :

- **max\_pasos**: Controla la cantidad máxima de iteraciones locales permitidas.
  - $N \geq 23$ :  $\text{max\_pasos} = 0$  (la búsqueda local se desactiva por completo para prevenir retrasos en sistemas masivos).
  - $20 \leq N < 23$ :  $\text{max\_pasos} = 100$  (se limita la iteración para proteger la CPU).
  - $N < 20$ :  $\text{max\_pasos} = 20$  (permite convergencia libre hasta el óptimo local).
- **permite\_swaps**: Booleano que habilita el vecindario de intercambio de elementos entre bloques.
  - Desactivado para  $N \geq 20$  ( $\text{permite\_swaps} = \text{False}$ ), limitando la búsqueda local a movimientos simples de inserción (*shift*) para mantener tiempos de respuesta de milisegundos.

### Configuración de la estrategia Branch and Bound en KGeoMIP

En el archivo [k\\_geometric.py](#), la estrategia `aplicar_estrategia_k_geometric` implementa una poda estructurada para acelerar el descarte de particiones que no tienen probabilidad de ser el óptimo global:

- **Cota de Pérdida (min\_emd)**: Se actualiza dinámicamente con el valor de la EMD de la mejor partición completa de tamaño  $k$  encontrada. Si el análisis de una rama intermedia reporta un coste superior a esta cota, la rama se descarta (break temprano).
- **Frontera de Búsqueda (frontera)**: Estructura de tipo lista ordenada que implementa la búsqueda priorizada. Se extraen continuamente los estados con menor coste de transición Hamming acumulado, garantizando que el algoritmo explore primero las regiones del espacio de estados más prometedoras.

### Modos de operación y límites físicos

| Modo de Operación                | Criterio de Activación       | Descripción Técnica                                         | Algoritmo Afectado |
|----------------------------------|------------------------------|-------------------------------------------------------------|--------------------|
| <b>Exhaustivo (Fuerza Bruta)</b> | $N \leq 8$ variables futuras | Evalúa el total de combinaciones del número de Stirling del | QNodes / KGeoMIP   |

|                              |                              |                                                                                                |         |
|------------------------------|------------------------------|------------------------------------------------------------------------------------------------|---------|
|                              |                              | segundo tipo de forma exacta.                                                                  |         |
| <b>Heurístico Submodular</b> | $7 \leq N \leq 22$ variables | Genera candidatos a partir de la estructura del árbol de fusiones Q y refina con VND.          | QNodes  |
| <b>Bypass Estructural</b>    | $N > 22$ variables           | Desactiva el árbol de fusiones Q. Genera y evalúa una sola partición estructural por balanceo. | QNodes  |
| <b>Modo Pequeño</b>          | $N < 20$ variables           | Calcula y memoiza los costos en todos los niveles Hamming de transición del hipercubo.         | KGeoMIP |
| <b>Modo Grande</b>           | $N \geq 20$ variables        | Calcula los costos de transición restringiéndolos únicamente a distancia Hamming 1.            | KGeoMIP |

## Automatización de pruebas masivas en Excel

### Estructura y lectura de Excel en QNodes (pruebas.xlsx)

El script carga la plantilla de entrada ubicada en src/results/pruebas.xlsx y lee la configuración de red y las variables del subsistema: \* **Configuración Superior (Filas 1 a 3):**

\* **Celda B1 (Estado inicial):** Estado activo/inactivo del sistema al inicio de la corrida (Ej: 1000000000 en formato de 10 bits).

\* **Celda B2 (Sistema):** El universo de variables consideradas (Ej: ABCDEFGHIJ).

\* **Celda B3 (Sistema Candidato):** Las variables que forman el subsistema causal.

\* **Lectura de Filas del Subsistema (Fila 5 en adelante):** El script está programado en Pandas para omitir las primeras 4 filas de cabecera (skiprows=4), detectando los nombres de columna en la fila 5:

\* **Columna A (#Prueba):** Índice o ID numérico del caso de prueba.

\* **Columna B (Alcance o Purview (t+1)):** Lista de letras de variables a evaluar en el futuro (Ej: ACEGI).

\* **Columna C (Mecanismo(t)):** Lista de letras de variables a evaluar en el presente (Ej: ACEGI).

**Conversión interna a binario:** Las cadenas de letras (como ACEGI) se procesan dinámicamente mediante la función `texto_a_binario(texto, universo)`. Si el universo es de 10 bits (ABCDEFGHIJ), ACEGI se convierte automáticamente en la máscara binaria "1010101010".

```

31 def lexio_a_binario(texto, universo):
32     return resultado
33
34
35
36
37
38
39
40
41 def iniciar():
42     print(
43         f"[{obtener_timestamp()}] Iniciando proceso de automatización estructurado..."
44     )
45     universo_letras = "".join([chr(65 + i) for i in range(LONGITUD_ELEMENTOS)])
46     ruta_excel = os.path.join("src", "results", "pruebas.xlsx")
47     ruta_salida = os.path.join("src", "results", "pruebas_con_resultados.xlsx")
48     if not os.path.exists(ruta_excel):
49         print(
50             f"[{obtener_timestamp()}] ✖ ERROR: No se encontró el archivo Excel en '{ruta_excel}'"
51         )
52         return
53     print(
54         f"[{obtener_timestamp()}] Cargando datos con Pandas para el procesamiento lógico..."
55     )
56     df_origen = pd.read_excel(ruta_excel, sheet_name=nombre_hoja, skiprows=4)
57     df_origen.columns = df_origen.columns.str.strip().str.replace("\n", " ")
58     print(
59         f"[{obtener_timestamp()}] Total de filas detectadas por Pandas: {len(df_origen)}"
60     )
61     print(f"[{obtener_timestamp()}] 📄 Abriendo plantilla con openpyxl...")
62     wb = openpyxl.load_workbook(ruta_excel)
63     if nombre_hoja not in wb.sheetnames:
64         print(f"[{obtener_timestamp()}] ✖ ERROR: La hoja '{nombre_hoja}' no existe.")
65         return
66     sheet = wb[nombre_hoja]
67     print(
68         f"[{obtener_timestamp()}] 🌐 Inicializando Gestor de Redes (Manager) a {LONGITUD_ELEMENTOS} bits..."
69     )
70     print(f"[{obtener_timestamp()}] 🚀 Inicializando Analizador de (KQNodes)...")

```

### ***Mapeo comparativo de lectura y escritura en hojas de cálculo***

Al inyectar los resultados de los experimentos (Partición, Pérdida y Tiempo de Cómputo), ambos algoritmos mapean de forma diferente sus celdas de destino en las respectivas plantillas Excel:

| Parámetro k               | Columnas<br><br>Resultantes en QNodes<br>(pruebas_con_resultados.xls<br>x) | Columnas<br><br>Resultantes en KGeoMIP<br>(DatosPruebas2026_1.xlsx) |
|---------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------|
| k=2 (Biparticiones)       | Columnas D, E, F<br><br>(Índices 4, 5, 6)                                  | Columnas K, L, M                                                    |
| k=3 (Tri-<br>particiones) | Columnas G, H, I<br><br>(Índices 7, 8, 9)                                  | Columnas Q, R, S                                                    |

|                     |                                          |                     |
|---------------------|------------------------------------------|---------------------|
| k=4 (4-Particiones) | Columnas J, K, L<br>(Índices 10, 11, 12) | Columnas Z, AA, AB  |
| k=5 (5-Particiones) | Columnas M, N, O<br>(Índices 13, 14, 15) | Columnas AI, AJ, AK |

La pérdida mínima de información se guarda en el Excel como un tipo float numérico nativo de Python, permitiendo al usuario realizar formateos numéricos personalizados de celdas o gráficos automáticos sin pérdida de precisión decimal.

---

## Optimización y consejos de rendimiento

Para optimizar el rendimiento al procesar bases de datos y plantillas masivas, considere los siguientes lineamientos:

1. **Gestión de Procesos en KGeoMIP:** El script de GeoMIP Método 2 utiliza la API de multiprocessing para aislar el cálculo de cada fila de Excel. Puede configurar el tiempo de timeout (línea 232 de src/main.py) para definir cuándo descartar filas atascadas (por defecto: timeout=3600 segundos).
2. **Caché en QNodes:** El framework QNodes utiliza internamente un diccionario de caché para bipartir\_y\_emd. Si realiza pruebas consecutivas del mismo subsistema dentro de un script avanzado, no olvide ejecutar `analizador.reset_estado()` para liberar la memoria caché acumulada y mantener el consumo de RAM bajo control.
3. **Caché NumPy (.npy):** Al cargar redes por primera vez, el proceso puede demorar varios segundos mientras lee el CSV plano. Una vez generado el archivo .npy correspondiente en la carpeta .samples/, asegúrese de no borrarlo para que las ejecuciones posteriores comiencen instantáneamente.

## 2.7 Solución de Problemas

Esta sección describe cómo resolver los incidentes más frecuentes al instalar, configurar y ejecutar las herramientas del proyecto.

### Errores Comunes en Consola

A continuación, se detallan los mensajes de error más comunes que pueden aparecer en la terminal, su significado y los pasos para solucionarlos:

1. **ValueError: Estado inicial debe tener longitud N**
  - *Significado:* El estado inicial ingresado por el usuario no tiene la misma cantidad de variables (o nodos) que la Matriz de Probabilidades de Transición (TPM).

- *Solución:* Verifique las dimensiones de la TPM. Si su red tiene  $N = 4$  nodos, el estado inicial de entrada debe ser una cadena de exactamente 4 caracteres binarios (por ejemplo, "1000").
- 2. **ValueError: No se encontró ninguna partición válida**
  - *Significado:* El algoritmo no pudo encontrar una división válida que asigne cada variable a un bloque sin dejar bloques vacíos en el futuro.
  - *Solución:* Asegúrese de que el valor del parámetro  $k$  (número de bloques) no sea mayor al número de variables activas del futuro ( $k \leq |A|$ ). Si tiene un subsistema de 3 variables en el futuro, no puede pedir una partición de  $k = 4$  bloques.
- 3. **FileNotFoundError: [Errno 2] No such file or directory: '...'**
  - *Significado:* El motor no encuentra el archivo CSV o NumPy .npz con la matriz de probabilidad en la ruta especificada.
  - *Solución:* Verifique que el archivo exista en el disco y que la ruta provista en la configuración sea la ruta absoluta correcta. En Windows, asegúrese de usar barras diagonales (/) o escapar las barras invertidas (\\).

**[!WARNING] Error de Codificación Unicode en Windows (Caracteres rotos):** Si al imprimir los resultados en consola observa caracteres extraños (como Ã¢ o cajas de error) en lugar de las llaves y corchetes, su consola de Windows está usando una codificación heredada (como CP1252). \* **Solución:** Ejecute sus scripts de Python anteponiendo la bandera `-X utf8` (ejemplo: `python -X utf8 exec.py`) o ejecute el comando `chcp 65001` en su terminal de PowerShell antes de iniciar Python.

## Problemas de Instalación y Dependencias

- **Error: ModuleNotFoundError: No module named 'numpy' o 'scipy'**
  - *Causa:* Las bibliotecas matemáticas esenciales no están instaladas en su entorno de Python.
  - *Solución:* Instale las dependencias ejecutando:

```
pip install numpy scipy pandas openpyxl colorama
```
- **Conflictos de Versión (NumPy 2.x):**
  - *Causa:* La versión instalada de NumPy es demasiado reciente y tiene incompatibilidades con el formato binario del código heredado de PyPhi.
  - *Solución:* Desinstale su versión actual e instale una compatible de la rama 1.x:

```
pip install "numpy<2.0.0"
```

**[!TIP] Uso de Entornos Virtuales:** Se recomienda encarecidamente utilizar un entorno virtual (como venv o conda) para evitar conflictos con las dependencias globales del sistema operativo:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

---

## Problemas de Ejecución y Rendimiento

### 4. El programa se congela o consume demasiada memoria RAM:

- *Causa:* Está intentando procesar un sistema extremadamente grande ( $N \geq 22$ ) sin optimizaciones de memoria en modo exhaustivo. La TPM determinista ocupa espacio exponencial ( $2^N \times N$ ). Para  $N = 25$ , almacenar la TPM en precisión estándar de 64 bits consume más de 6 Gigabytes de RAM.
- *Solución:*
  - **Para KQNodes:** Asegúrese de activar el bypass automático para sistemas grandes ( $N > 22$ ).
  - **Para KGeoMIP:** Verifique que el sistema esté operando en **modo restringido** (el cual solo calcula la tabla para Hamming 1).
  - **Casteo a Tipo Compacto:** Asegúrese de que el cargador (Manager) guarde la TPM en formato de baja precisión (np.int8 para redes deterministas o np.float32 para estocásticas).

### 5. El cálculo de EMD tarda varios minutos:

- *Causa:* Está evaluando la Earth Mover's Distance (EMD) mediante solvers lineales tradicionales en cada paso de la búsqueda local.
  - *Solución:* Active la **optimización Slice-First** en KQNodes y las cachés locales (\_bipartir\_emd\_cache). Estas reducen la resolución multidimensional de EMD a una sumatoria lineal de diferencias de vectores de probabilidad marginales reducidos.
- 

## Diagnóstico de Datos de Entrada Problemáticos

El formato de las matrices de entrada debe ser rigurosamente validado para evitar cálculos inconsistentes:

- **Filas que no suman 1.0 (Matrices Estocásticas):**
    - *Problema:* En una TPM estocástica, la suma de las probabilidades de transición de cada fila debe ser estrictamente 1.0. Si no es así, el modelo de estados será probabilísticamente inválido.
    - *Corrección:* Utilice el script de normalización utilitario antes de cargar la red, o configure el cargador para que avise si alguna fila tiene una desviación superior a  $10^{-5}$ .
  - **Formatos CSV Inconsistentes:**
    - *Problema:* El archivo CSV utiliza comas, como separador pero el sistema espera dos puntos : o punto y coma ; debido a configuraciones regionales de Windows.
    - *Corrección:* Abra su archivo de configuración pyphi\_config.yml y verifique que el delimitador (delimiter) coincida con el delimitador de su archivo CSV de entrada.
- 

## 2.8 Referencia Rápida

Esta sección es una guía resumida para consulta de parámetros y comandos de uso cotidiano.



## Comandos de Ejecución Clave

A continuación, se enumeran los comandos para ejecutar los scripts del proyecto en la consola:

| Comando                        | Sintaxis / Ejemplo                       | Descripción                                                                                                               |
|--------------------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Ejecución Unificada</b>     | <code>python -X utf8 exec.py</code>      | Abre el menú interactivo para seleccionar la estrategia (KGeoMIP/KQNodes), cargar la TPM y configurar parámetros.         |
| <b>Buzón por Lotes (Excel)</b> | <code>python -X utf8 exec_user.py</code> | Ejecuta la suite de pruebas automatizada, procesa las 49 configuraciones del Excel pruebas.xlsx y escribe los resultados. |
| <b>Pruebas de Correctitud</b>  | <code>python -m pytest tests/</code>     | Ejecuta la suite de tests unitarios y de integración para validar la correctitud del motor algebraico.                    |

## Tabla de Parámetros y Configuraciones

Esta tabla detalla las variables que el usuario puede ajustar para controlar el comportamiento de la búsqueda:

| Parámetro        | Tipo   | Valor por Defecto | Rango / Valores Válidos              | Descripción                                                                                        |
|------------------|--------|-------------------|--------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>k</b>         | Entero | 3                 | 3 a 5                                | Número de bloques en los que se dividirá el subsistema dinámico.                                   |
| <b>N</b>         | Entero | -                 | 1 a 25                               | Dimensión total (número de variables) del sistema dinámico.                                        |
| <b>condicion</b> | Cadena | "111...1"         | Cadena de bits (0 y 1) de longitud N | Define las dimensiones que se van a condicionar a su estado inicial. Los bits en 0 se condicionan. |

|                  |        |           |                                        |                                                                             |
|------------------|--------|-----------|----------------------------------------|-----------------------------------------------------------------------------|
| <b>alcance</b>   | Cadena | "111...1" | Cadena de bits (0 y 1) de longitud $N$ | Delimita las variables del futuro ( $V_{t+1}$ ) que componen el subsistema. |
| <b>mecanismo</b> | Cadena | "111...1" | Cadena de bits (0 y 1) de longitud $N$ | Delimita las variables del presente ( $V_t$ ) que componen el subsistema.   |

## Formato de Archivos de Entrada y Salida

### 1. Archivo de Entrada (TPM)

- **Formato:** CSV (valores separados por delimitador).
- **Dimensiones:** La matriz debe ser de tamaño  $2^N \times N$ , donde cada fila representa el estado conjunto del presente del sistema en orden binario, y cada columna representa la probabilidad de que la variable correspondiente esté en estado activo (1) en el instante futuro  $t + 1$ .
- **Ejemplo de fila ( $N = 3$ ):**  
0.1 : 0.8 : 0.05

### 2. Archivo de Salida (Resultados en Excel)

- **Formato:** Libro de Excel (.xlsx).
- **Columnas rellenas por el sistema:**
  - *Partición de Mínima Información (MIP):* Expresada en notación de bloques de texto (por ejemplo, AB/C para  $k = 2$ , o A/B/C para  $k = 3$ ).
  - *Pérdida Mínima ( $\Phi$ ):* Valor decimal representando la pérdida de información integrada del sistema particionado.
  - *Tiempo de Ejecución:* Duración del procesamiento de la fila expresada en segundos con cuatro decimales (ejemplo: 0.0182s).

## Glosario de Términos

- **Partición de Mínima Información (MIP):** Es la división de un sistema dinámico en bloques disjuntos que provoca la menor pérdida de información integrada al asumir la total independencia de sus bloques.
- **Información Integrada ( $\Phi$ ):** Medida de la sinergia informacional de un sistema. Cuantifica cuánta información genera el sistema conjunto por encima de la suma de sus partes independientes.
- **Earth Mover's Distance (EMD):** Métrica de distancia de transporte óptimo que calcula el costo mínimo para transformar una distribución de probabilidad en otra. Se utiliza para medir la diferencia entre la TPM real y la TPM particionada.

- **Marginalización:** Proceso algebraico mediante el cual se promedian las probabilidades de variables que no son de interés en el análisis actual, reduciendo la dimensionalidad del tensor.

**Slicing (Rebanado):** Operación que consiste en seleccionar una cara o subconjunto del tensor de probabilidades fijando una variable a su estado inicial (condicionamiento).